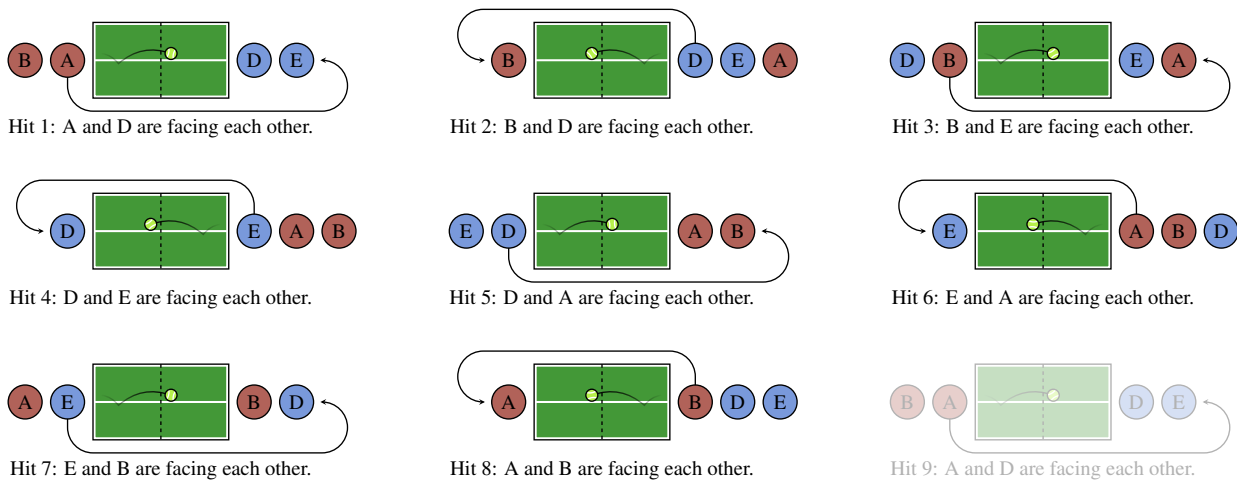


Problem A. Around the Table

Source file name: Around.c, Around.cpp, Around.java, Around.py
Input: Standard
Output: Standard

Emilia and her brother Alex meet up with a lot of friends in the park to play table tennis together. Since there is only one table, they play round-the-table which is everybody's favourite.

The rules are simple: There are $\ell \geq 2$ kids queueing up on the left side and $r \geq 1$ kids queueing up on the right side of the table. The first kid on the left side starts with the ball. Whoever has the ball hits it over the net onto the opposite side and runs around the table to join the queue at the opposite side. This is repeated until the first mistake.



Visualization of the first example. During Hit 1 and 5, and during Hit 3 and 7, we observe the same pairing. After 8 hits we end up in the initial state again and cannot observe any new pairing afterwards. Therefore, there are 6 different pairings we will see in total.

Emilia, Alex, and their friends play this game all the time and are so good that they can go on almost forever without making any mistake. After some time, Emilia notices that she has never faced Alex, meaning that they have never been at the front of opposite queues at the same time. She wonders if this will ever happen in this round. Curious about this, she starts keeping track which pairings have already faced each other.

How many different pairings has she counted after the ball went over the net 10^{100} times?

Input

The input consist of:

- One line with two integers ℓ and r ($2 \leq \ell \leq 10^9$ and $1 \leq r \leq 10^9$), the number of players on the left and right side of the table, respectively.

Output

Output the number of different pairings Emilia counts.



Example

Input	Output
2 2	6
2 3	10
3 2	5
5 2	14

Problem B. Bustling Busride

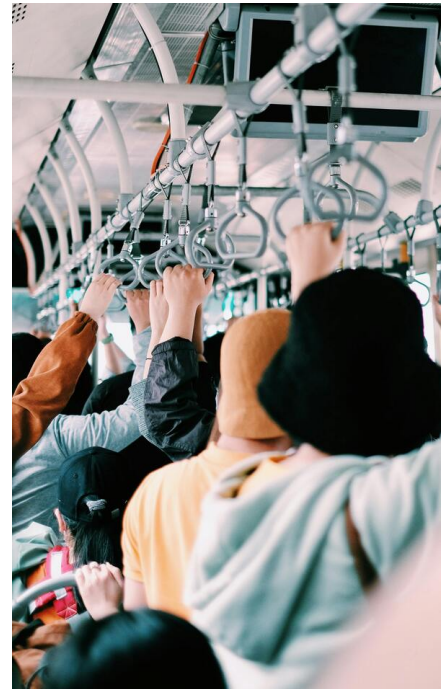
Source file name: Bustling.c, Bustling.cpp, Bustling.java, Bustling.py
Input: Standard
Output: Standard

The university of Bithamptonis served by exactly one bus line. On its way to the city centre, it serves several stops at which students may exit. Every student has a fixed bus stop where they want to exit.

It is Friday afternoon, 4 PM, and as always, all the students want to go home, leading to quite a long queue at the bus stop. Fortunately, the bus line is served in regular intervals, with the first bus arriving at 4 PM.

Whenever a bus arrives at the university, everyone in the queue tries to enter the bus, which makes the buses very crowded. This has led to numerous complaints where people tried to exit buses but were unable to because of the sheer amount of people. As a consequence, the bus company decided that at every stop which someone in the bus has as destination, everyone in the bus must exit it. Those who want to travel further enter again. For every time a passenger enters or exits the bus, the bus needs to wait w seconds.

To offer the best service, the bus company wants to minimize the maximum time it takes anybody from 4 PM until they reach their destination. For each bus, the bus driver can decide how many people from the front of the queue enter the bus. The number of people that can enter a bus is unlimited. Help the bus drivers make the optimal decisions to achieve the company's goal.



A crowded bus. By: Pexels/Pew Nguyen

Input

The input consists of:

- One line with four integers n , b , r , and w ($1 \leq n, b \leq 10^5$, $1 \leq r, w \leq 10^6$), the number of passengers, the number of bus stops, the time between the arrival of two buses at the university, and the delay for exiting and entering.
- One line with b integers d_i ($1 \leq d_i$, $\sum d_i \leq 10^6$), the travel time from the $(i-1)$ th bus stop to the i th bus stop (the 0th bus stop is the university).
- One line with n integers t_i ($1 \leq t_i \leq b$), indicating that the destination of the i th person in line is the t_i th bus stop.

All times are given in seconds.

Output

Output the minimum number of seconds until every person in line has exited at their destination.

Example

Input	Output
3 3 20 1 2 2 2 2 3 1	18
4 2 1 10 3 2 1 2 2 1	27
5 3 3 3 2 2 1 3 3 2 1 1	17

Explanation

In the first example, it is optimal to let everyone board the first bus. The first boarding takes 3 seconds. Then, the bus drives for 2 seconds to get to the bus stop 1. After 3 seconds, everybody exited and after another 2 seconds, the people continuing their journey are back on the bus. After 2 more seconds, they arrive at the next stop where it takes 2 seconds for everybody to exit and one second for the remaining person to get back on the bus. After 2 more seconds, the bus arrives at the final destination where it takes one second for the last person to exit.

In the second example, it is optimal to use one bus per person.

In the third example, it is optimal to assign the first two passengers to one bus. Everybody else gets their own bus.

Problem C. Congklak

Source file name: Congklak.c, Congklak.cpp, Congklak.java, Congklak.py
 Input: Standard
 Output: Standard

Alice and Bill really enjoy playing board games, and they are always looking for new challenges. Recently, they discovered the Indonesian game *Congklak* which is played with a game board made up of several holes containing some number of stones. After playing some games, Alice quickly got the hang of it and won every game, so Bill did not want to play any more. Instead, inspired by the rules of the game, he came up with the following challenge for Alice:

There is a game board with n holes arranged in a long row. These holes are numbered from 1 to n from left to right. Initially the i th hole contains a_i stones. Note that this setup differs from the usual Congklak game, where the game board consists of two rows and one large hole at each end.

Now Bill will play t games where each game goes as follows:

Bill starts the game at the first hole, holding one new stone in his hand. He then moves along the game board from hole 1 to hole n . At each hole i , he first checks how many stones are currently in the hole, and depending on the result he performs exactly one of the following two actions:

- i) If the hole is empty, he drops one stone into it. Next, he checks how many stones are still in his hand. If his hand is empty, the game stops. Otherwise, he moves his hand to hole $i + 1$ next and repeats the steps.
- ii) If there is at least one stone in the hole, he also drops one stone into it. Next, he checks how many stones are still in his hand. If his hand is empty, he takes out all the stones from hole i into his hand. Regardless of whether or not his hand was empty, he moves his hand to hole $i + 1$ next and repeats the steps.

When Bill moves his hand past hole n , the game stops and Bill discards any stones that he still holds in his hand.

Bill challenges Alice to predict in advance the number of stones in every hole after playing exactly t games. Note that the game board is **not** reset after playing a game, i.e. the initial configuration of the second game is the same as the configuration when the first game ends.

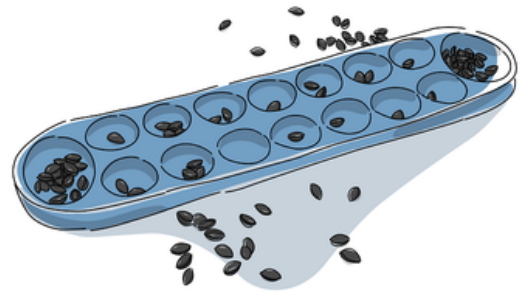
Input

The input consists of:

- One line with two integers n and t ($1 \leq n \leq 10^5$, $1 \leq t \leq 10^{12}$), the number of holes and the number of games.
- One line with n integers $a_1, \dots, a_n$ ($0 \leq a_i \leq 10^{12}$), where a_i describes the initial number of stones in the i th hole.

Output

Output n integers, the i th of which is the number of stones in hole i after playing t games.



A typical Congklak game board.
 CC BY-SA 4.0 by rikadiani17 on Pixabay

**Example**

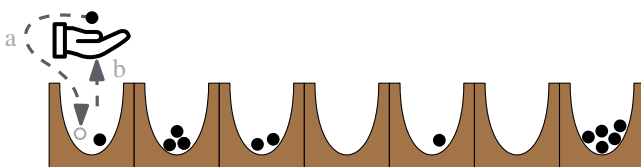
Input	Output
7 1 1 3 2 0 1 0 5	0 4 0 1 2 1 5
4 4 10000000000000 1 2 3	1 3 0 5

Explanation

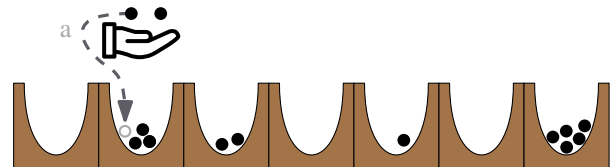
In the first example, Bill plays exactly one game. The figure below visualizes the steps performed during this game.

In the second example, the initial number of stones per hole is [1000000000000, 1, 2, 3]. The number of stones per hole after each game is:

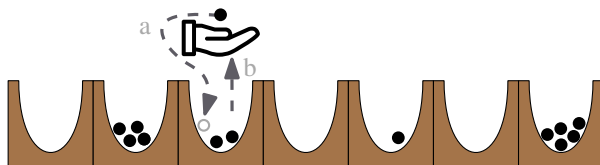
1. [0, 2, 3, 4]
2. [1, 2, 3, 4]
3. [0, 3, 0, 5]
4. [1, 3, 0, 5]



1. Hole 1 is not empty (case ii).
 - (a) Bill drops one stone into hole 1.
 - (b) Bill's hand is empty now, so he takes all two stones out of hole 1 and continues to the next hole.



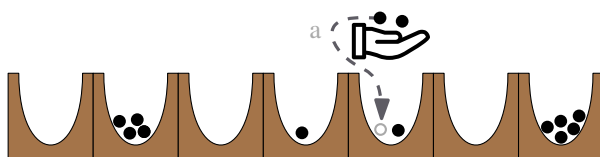
2. Hole 2 is not empty (case ii).
 - (a) Bill drops one stone into hole 2.
 - (b) Bill still holds one stone, so he just continues to the next hole.



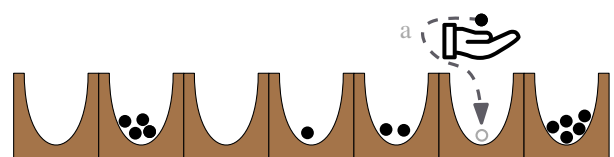
3. Hole 3 is not empty (case ii).
 - (a) Bill drops one stone into hole 3.
 - (b) Bill's hand is empty now, so he takes all three stones out of hole 3 and continues to the next hole.



4. Hole 4 is empty (case i).
 - (a) Bill drops one stone into hole 4.
 - (b) Bill still holds two stones, so he just continues to the next hole.



5. Hole 5 is not empty (case ii).
 - (a) Bill drops one stone into hole 5.
 - (b) Bill still holds one stone, so he just continues to the next hole.



6. Hole 6 is empty (case i).
 - (a) Bill drops one stone into hole 6.
 - (b) Bill's hand is empty now, so the game stops.

Visualization of the only game which Bill plays in the first example.

Hand icon: MIT Licence by jtblabs on svgrepo



Problem D. Dry Land

Source file name: Dryland.c, Dryland.cpp, Dryland.java, Dryland.py
Input: Standard
Output: Standard

There are a lot of wetlands in Florida and finding space for building houses is a challenge. This problem is even more serious when people insist their houses to be rectangular shapes.

Let's assume our land is a two-dimensional (rectangular) grid and each grid cell is either wet or dry. We are interested in finding the largest rectangular dryland, i.e., we want to build the largest house (in area). Note that the house must be dryland in rectangular shape, i.e., the dry cells (area to build the house) must be rectangular and there can't be any wet cells in that rectangle.

Input

The first input line contains two integers: r ($1 \leq r \leq 200$), indicating the number of rows in our land and c ($1 \leq c \leq 200$), indicating the number of columns in our land. Each of the next r input lines specifies a row in the land; each line contains c characters (starting in column 1), each character being either 1 (dryland) or 0 (wetland).

Output

Print the area of the largest rectangular dryland.

Example

Input	Output
4 5 10010 01111 11110 00001	6
3 2 10 11 11	4
2 2 10 01	1

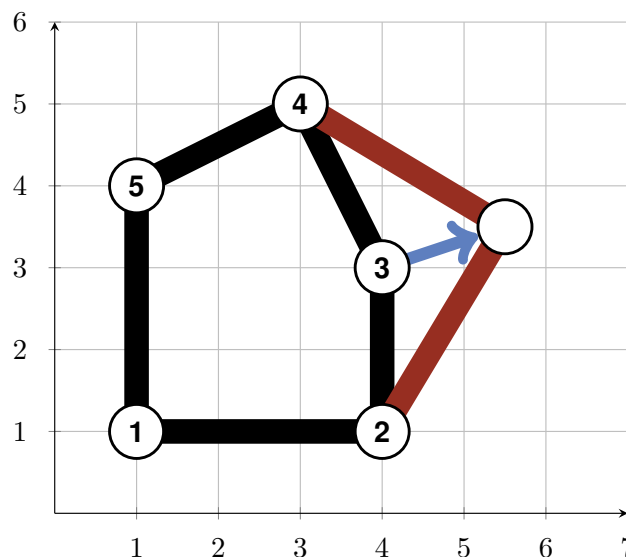
Problem E. Engineering Excellence

Source file name: Engineering.c, Engineering.cpp, Engineering.java, Engineering.py
 Input: Standard
 Output: Standard

You are the engineer in charge of designing a wheel for a new space rover. As you do not have enough time to reinvent the wheel, you decide to copy your predecessor's work and make only one small change.

Looking at the plan, you notice that your predecessor's wheel has the shape of a convex polygon for structural reasons. It is well known that wheels with a larger perimeter roll farther per rotation, so surely they must be superior. You attempt to increase the perimeter by as much as possible by moving a single point on the outside of the wheel. While experimenting, you notice that wheels do not seem to work if they are non-convex or if there is an internal angle below 90 degrees.

What is the maximum possible achievable increase in the perimeter of the wheel without violating the above restrictions?



Visualization of the first example. Point 3 is moved to (5.5, 3.5), increasing the perimeter by ≈ 1.59488 .

Input

The input consists of:

- One line with an integer n ($4 \leq n \leq 10^5$), the number of points of the wheel.
- n lines, each with two integers x and y ($|x|, |y| \leq 10^5$), the coordinates of the points.

The points are given in counterclockwise order and form a convex polygon with no internal angle below 90 degrees. Note that the convex polygon may contain **collinear** points, but no two points are at the same position.

Output

Output the maximum possible absolute increase of the perimeter of the wheel.

Your answer should have an absolute or relative error of at most 10^{-6} .



Example

Input	Output
5 1 1 4 1 4 3 3 5 1 4	1.594883917346
7 0 -4 3 -3 5 -2 3 2 -6 -2 -6 -4 -2 -4	3.151142198204
4 0 0 1 0 1 1 0 1	0.000000000000

Problem F. Fair and Square

Source file name: Fair.c, Fair.cpp, Fair.java, Fair.py
 Input: Standard
 Output: Standard

Felix has ordered a large rectangular pizza for his birthday party. He only quickly went to the kitchen to grab some plates and cutlery, and when he came back his guests had already started helping themselves to various parts of the pizza. The pizza, which originally was made up of $h \times w$ square pieces of equal size, is now missing some of these pieces:

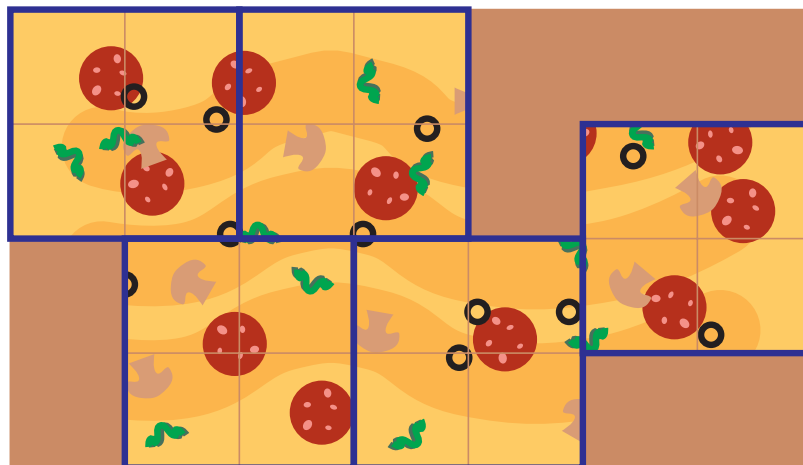


Illustration of the first example case, with a division into squares of side length 2.

To ensure that the distribution of the rest of the pizza proceeds in a much more orderly fashion, Felix wants to divide up the remaining pizza into larger square shaped areas. Felix can only cut along the grid lines and cannot rearrange any parts of the pizza. Each square should have the same side length, which should be as large as possible in order to minimize the number of plates needed.

Find this largest possible side length. Note that an answer always exists because the pizza can always be divided into 1×1 squares.

Input

The input consists of:

- One line with two integers h and w ($1 \leq h, w \leq 2500$), the height and width of the grid.
- h lines, each with a string of length w consisting of '#' (denoting pizza pieces that are still there) and '.' (denoting pizza pieces that have already been taken).

It is guaranteed that the input contains at least one '#'.

Output

Output an integer, the largest possible side length such that the remaining pizza can be divided into squares of that side length.



Example

Input	Output
4 7 ####... ####.## .##### .####..	2
3 5 ##### ##### ###..	1
3 5 .##.. ##### ##.##	1
5 13 ...###..... ...###.###. .#####.###. .###.....###. .###.....	3



Problem G. Farmers' Market

Source file name: Farmers.c, Farmers.cpp, Farmers.java, Farmers.py
Input: Standard
Output: Standard

Dr. Meade went to the local Farmers' Market to get some fresh fruits. Most stores expected the customers to put fruits in bags and then the customers would be charged based on how much each bag weighs. Dr. M was exhausted from grading the final exams and, fortunately, he found a store where apples were already prepackaged!

All bags have the same number of apples (say, 5 apples in each bag) and you pick as many bags as you want. But, rather than charging you by the number of bags or their weights, the store charges you by how many dozens of apples you have! If your total count is not a multiple of dozens, you are still charged for dozens. For example, if each bag has 5 apples and you pick 4 bags (i.e., 20 apples total), you are charged for 24 (two dozens) apples since 20 is not a multiple of dozens.

Given how many apples there are in each bag, number of bags Dr. M has picked and the cost for a dozen of apples, determine how much Dr. M has to pay.

Input

The first input line contains two integers: a ($1 \leq a \leq 20$), indicating the number of apples in each bag and b ($1 \leq b \leq 20$), indicating how many bags Dr. M has picked. The second input line contains an integer, d ($1 \leq d \leq 20$), indicating the cost for a dozen of apples.

Output

Print how much Dr. M has to pay.

Example

Input	Output
5 4 10	20
5 6 15	45
16 3 20	80



Problem H. Basketball Score

Source file name: Basketball.c, Basketball.cpp, Basketball.java, Basketball.py
Input: Standard
Output: Standard

There are three types of shots in basketball: 1 pointer (free throw), 2 pointers, and 3 pointers.
Given how many shots of each type two teams made, determine the winner.

Input

The first input line contains three integers (each between 0 and 99 inclusive), indicating how many shots of each type Team1 made. The first integer indicates 1-pointer shots, the second integer indicates 2-pointer shots, and the third integer indicates 3-pointer shots.

The second input line provides the shots made by Team2, following the same format as the first input line.

Output

Print 1, 2 or 0 (zero), indicating which team won or if the game was a tie. The team with higher score (total points) wins. If both teams have the same total points, then the game is a tie.

Example

Input	Output
10 20 30 50 10 10	1
5 5 5 1 99 1	2
1 2 3 0 1 4	0

Explanation

Explanation of the First Example Input/Output:

Score (total points) for Team1 is 140 and for Team2 is 100, so Team1 wins.

Problem I. Island Urbanism

Source file name: Island.c, Island.cpp, Island.java, Island.py
 Input: Standard
 Output: Standard

On a small volcanic island far away from any major landmass, the *Grand Council for Painless Cycling* (GCPC) is facing regular complaints about the bad quality of the bike path network. Their budget is limited, but they would like to improve the situation for all the cyclists on their island. A survey helped to determine the most common destinations of the cyclists. The GCPC expects that the cyclists are satisfied with the bike path replacements if there is a way to cycle between any two destinations using only newly replaced bike paths. In order not to unduly favour any of the villages, the Council has decided that a maximum of 7 destinations per village should be taken into account.

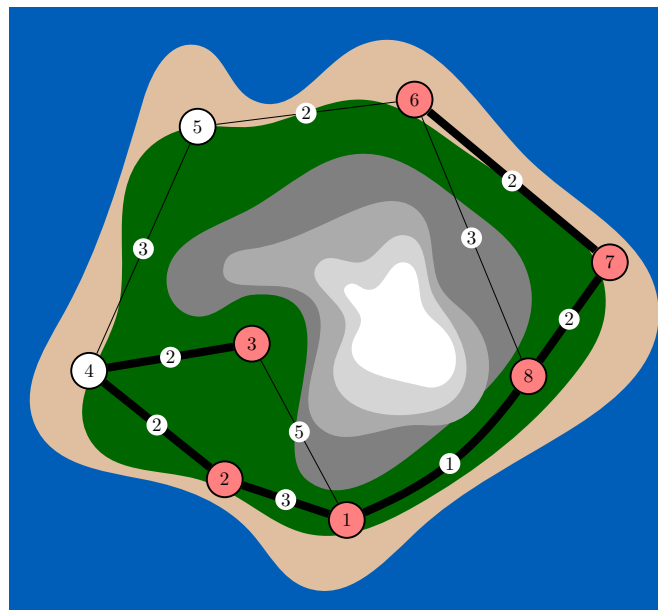


Illustration of the second example. Village 1 consists of Junctions 1, 2, 3, and 4, Village 2 consists only of junction 5, and Village 3 consists of the remaining junctions. Junctions that are destinations are coloured red.

There is one main cyclic bike path going around the volcano. On its way, it traverses every village on the island exactly once. Each village may have additional bike paths. How much does the GCPC have to spend at least in order to replace bike paths to connect all destinations via new paths?

Input

The input consists of:

- One line with four integers n , m , v and k ($3 \leq n \leq 5000$, $n \leq m \leq 20\,000$, $3 \leq v \leq n$, $1 \leq k \leq n$), the number of junctions, the number of bike paths, the number of villages, and the number of destinations.
- One line with v integers u_i ($1 \leq u_i$, $\sum_{i=1}^v u_i = n$), the i th of which describes the number of junctions in village i .
- m lines each containing three integers a , b and c ($1 \leq a, b \leq n$, $1 \leq c \leq 10^9$), describing a bidirectional bike path between junction a and junction b with a replacement cost of c .

- One line with k distinct integers t ($1 \leq t \leq n$), the junctions that have to be connected by the replaced bike paths.

Additionally, the following is guaranteed:

- Village 1 consists of junctions $1, \dots, u_1$, village 2 consists of junctions $u_1 + 1, \dots, u_1 + u_2$, and so on. The last village consists of junctions $1 + \sum_{i=1}^{v-1} u_i, \dots, n$. Within each village, it is possible to travel between any pair of junctions without leaving the village.
- There is a bike path between junctions $\sum_{i=1}^j u_i$ and $1 + \sum_{i=1}^j u_i$ for each $1 \leq j \leq v - 1$, as well as a bike path between junctions n and 1 . There are no other bike paths between different villages.
- There is at most one bike path between each pair of junctions and no bike path connects a junction to itself.
- In each village there are at most 7 junctions that need to be part of the improved cycling network.

Output

Output the minimum cost which allows replacing bike paths such that every trip between any two destinations can be done using only newly replaced paths.

Example

Input	Output
3 3 3 3 1 1 1 2 1 5 2 3 4 1 3 3 2 1 3	7
8 10 3 6 4 1 3 1 2 3 2 4 2 1 3 5 3 4 2 4 5 3 5 6 2 8 6 3 6 7 2 7 8 2 8 1 1 2 3 1 7 6 8	12



Problem J. Injured Shoulder

Source file name: Injured.c, Injured.cpp, Injured.java, Injured.py
Input: Standard
Output: Standard

Dr. Orooji injured his right shoulder a few months ago and could type with only one hand for a while. For some reasons, Dr. O missed the space character (blank) often and two words would be next to each other without the space (i.e., words concatenated) and the spell-checker would complain.

Given the words in a dictionary and a word Dr. O has typed, determine the status of the typed word.

Input

The first input line contains an integer, n ($2 \leq n \leq 20$), indicating the number of words in the dictionary. Each of the next n input lines contains a dictionary word. Each word starts in column 1, contains at least one lowercase letter, at most 20 lowercase letters, and no other characters. Assume that the dictionary words are unique, i.e., no duplicates.

The next input line contains an integer, m ($1 \leq m \leq 100$), indicating the number of words Dr. O has typed. Each of the next m input lines contains a typed word. Each typed word starts in column 1, contains at least one lowercase letter, at most 40 lowercase letters, and no other characters.

Output

For each typed word, output one line as follows:

- 1 – If the typed word is in the dictionary.
- 2 – If the typed word is a concatenation of two words in the dictionary, i.e., Dr. O missed typing the space character (blank).
- 0 (zero) – Otherwise.

Note that if a typed word is in the dictionary and it is also a concatenation of two words in the dictionary, output should be 1.

(Example Input/Output on the next page)



Example

Input	Output
7	1
this	1
is	2
a	1
test	1
th	2
the	0
e	2
12	0
is	0
test	1
isthis	2
the	
th	
ee	
eee	
thee	
best	
istestis	
e	
ea	



Problem K. Income Inequality

Source file name: Income.c, Income.cpp, Income.java, Income.py
Input: Standard
Output: Standard

According to Wikipedia, in 2021, the top 1% of households own 32.3% of the wealth in the United States. More generally, for any given percentage x , we could look at the data and state that the top $x\%$ of households own $y\%$ of the wealth.

It's obvious that for all societies with some inequality, it is always the case that $y > x$, except for $x = 0$ and $x = 100$. Of all possible choices for x , what is the maximum value of $y - x$?

Given the incomes of all people in a society, determine the maximum value of $y - x$ where the top $x\%$ of the people have $y\%$ of the society's wealth.

Input

The first input line contains a single integer, n ($2 \leq n \leq 10^6$), indicating the number of people in the society.

The second input line contains n integers; each integer, m ($1 \leq m \leq 10^{12}$), provides the wealth of a household in the society. Please note that the income values are not necessarily distinct.

Output

Print the maximum possible value of $y - x$, where the top $x\%$ of households in the society own $y\%$ of the society's wealth. Any answer within an absolute or relative error of 10^{-6} will be accepted.

Example

Input	Output
4 11 1 2 6	35.000000
5 35 25 30 60 50	15.000000

Problem L. Labour Laws

Source file name: Labour.c, Labour.cpp, Labour.java, Labour.py
Input: Standard
Output: Standard

The “Arbeitszeitgesetz” (ArbZG) regulates the work and break times of employees in Germany. According to this law, if an employee works more than 6 hours on a given day, the employee **must** take a break of at least half an hour during their shift. If they work for more than 9 hours, their break must be at least 45 minutes. Additionally, working for more than 10 hours on a day is strictly forbidden.

Alexander works at his company’s HR department. As the working hours at his company are very flexible, the employees are allowed to work as many hours or as few as they want (within the legal constraints of course). The monthly salary for each employee thus depends on the time spent working. Therefore, one of Alexander’s monthly tasks is to collect the times each employee has worked on any given day, so that he can calculate and pay out the employees salaries every month.

Unfortunately, today one of his colleagues forgot to record her lunch break and cannot remember how much time she spent on her break. As Alexander does not want to put his colleague at a disadvantage, he wants to use the legally required minimum break time for his calculations.

While calculating the total time his coworker spent at work is easy, correctly calculating the legally required minimum break time can be tricky. Can you help Alexander determine the minimum time his coworker must have spent at lunch today? Note that the time spent at work is the time spent working plus the time spent in break.

Input

The input consists of:

- One line with an integer t ($0 \leq t \leq 1440$), the number of minutes spent at work today.

Output

Output a single non-negative integer, the minimum number of minutes spent in break to make the work time legal.

Example

Input	Output
495	30
360	0
540	30
0	0



Some workplaces use dedicated machines to record work time. In Germany, these are called “Stempeluhren”. CC BY-SA 4.0 by u_h0yvbj97 on Pixabay

Problem M. Mex Hex

Source file name: Mexhex.c, Mexhex.cpp, Mexhex.java, Mexhex.py
Input: Standard
Output: Standard

The peace and quiet of your home city is being disturbed by the loud purrs and nightly creeping of a nearby magic bobcat. The mayor is sending you to scare it off – you are the city’s Guardian for Collateral, Personal, and other Catastrophic Damages after all. You take on the challenge and prepare for scaring away the magnificent animal.

To defend itself against you, the magic bobcat (as the name suggests) tries to hurt you in unconventional ways. When it attacks, it casts a number of magic *spells*. Each spell is cast with a particular *potential*, which can be expressed as a natural number.¹ If you get hit with spells that have the potentials $p_1, p_2, \dots, p_n$, then the pain that you will feel from them is $\text{mex}(\{p_1, p_2, \dots, p_n\})$, where mex denotes the Minimum Excluded Value.² Note that the spells do not hurt you immediately. Only after all spells are cast will you feel the pain based on the mex of their potentials.

To protect yourself, you bring a *shield* which can, also through magic, deflect the spells. Your shield has an *activation duration* d , which means that when you activate the shield, the next d spells do not hit you. Afterwards, the shield must regenerate its magic and cannot be activated for the following d spells. Apart from that, you can activate the shield as often as you want. To ensure that you give the magic bobcat a good scare, the mayor requested that you sneak up to it. As the bobcat would spot the glow of your activated magic shield, the earliest time you can activate the shield is when you stand before the culprit, right before it casts the first spell.



A regular (non-magic) bobcat.
Photo by Becker1999

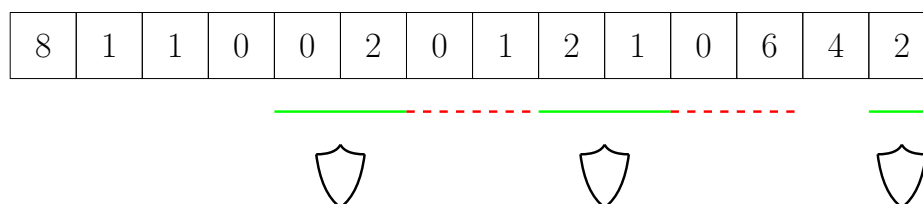


Illustration of Example 3. The numbers in the boxes indicate the potentials of the spells. In this example, your shield’s activation duration is $d = 2$. By activating the shield right before the 5th, 9th, and 14th spell, you block the spells underlined in green. You cannot activate your shield for the spells that are underlined with a dashed red line, because it is regenerating its magic at these times.

You must now devise a tactic to sustain as little pain as possible. From studying the previous encounters with magic bobcats, you know the potentials of the spells that will be cast against you. What is the minimum pain you can receive from them if you activate your shield optimally?

Input

The input consists of:

- One line with two integers n and d ($1 \leq n \leq 10^5$, $1 \leq d \leq n$), the number of spells and the activation

¹For the purposes of this task, the natural numbers are defined as $\mathbb{N} = \{0, 1, 2, \dots\}$.

²Given a set $S \subseteq \mathbb{N}$, $S \neq \mathbb{N}$, $\text{mex}(S)$ is the smallest number $m \in \mathbb{N}$ that is *not contained* in S .

duration of your shield.

- One line with n integers $p_1, \dots, p_n$ ($0 \leq p_i < n$), the potentials of the n spells.

Output

Output a single integer, the minimum pain you can receive from the spells.

Example

Input	Output
5 1 0 1 0 1 0	0
5 2 0 1 0 1 0	2
14 2 8 1 1 0 0 2 0 1 2 1 0 6 4 2	2
4 2 0 1 2 0	1

Explanation

In the first example, you can activate the shield for the first spell, let it regenerate its magic for the second, activate it again for the third, regenerate for the fourth, and activate again on the fifth spell. This means that only the two spells of potential 1 hit you. As $\text{mex}(\{1, 1\}) = 0$, you receive 0 pain.

In the second example, no matter how you activate your shield, during the time it regenerates, a spell of potential 0 and a spell of potential 1 hit you. Thus, the mex of the spells that hit you is at least 2. This amount of pain can be achieved by not using your shield at all.

In the third example, use the shield as depicted in Figure 6. Then the spells that hit you have potentials 8, 1, 1, 0, 0, 1, 0, 6, and 4. The mex of these potentials is 2. It can be shown that there is no way of activating your shield so that the mex of the potentials of the remaining spells is 0 or 1.

In the fourth example, recall that you cannot activate your shield any earlier than right before the first spell. Thus, you cannot block both spells of potential 0. Instead, by blocking the second and third spell, all spells that hit you have potential 0.